

Quality Lab Manual



Operating System Lab 2ICPC204

Experiment No. 5

**Program based on Non – Preemptive CPU
Scheduling Algorithms.**

Course Instructor -

PROF.MADHURI J SIDDHARAPU
ASSISTANT PROFESSOR

Experiment No. 5

Expt No: 5 (a)

Title of Experiment: Program based on Non – Preemptive CPU Scheduling Algorithms.

Aim of Experiment: Write a C program to implement First Come First Serve (FCFS) scheduling algorithm.

System Requirements – Operating System: Microsoft Windows 7 or later (Windows 10/11 recommended for compatibility).
Compiler: Turbo C version 2.0 or later.
Hardware: Minimum 512 MB RAM and a basic CPU (e.g., Intel 80386 or later).
Disk Space: Approximately 5 MB for Turbo C and associated files.

Experiment Outcomes –

1. Understanding of different CPU scheduling algorithms, including FCFS, and SJF Scheduling, and how each one affects process management in an operating system.
2. Learn to analyze the trade-offs between different algorithms, such as the simplicity of FCFS versus the efficiency of SJF and how each algorithm impacts metrics like waiting time and turnaround time.
3. Develop practical programming skills in C and gain experience with handling scheduling-related tasks and logic in a real-world context.
4. Learn to choose appropriate scheduling algorithms based on specific system requirements and workload characteristics, thus understanding how to apply theoretical concepts to practical scenarios.

First come first serve (FCFS) scheduling algorithm simply schedules the jobs according to their arrival time. In this type of algorithm, processes which requests the CPU first get the CPU allocation first. As the process enters the ready queue, its PCB (Process Control Block) is linked with the tail of the queue and, when the CPU becomes free, it should be assigned to the process at the beginning of the queue.

Advantages of FCFS:

1. The simplest form of a CPU scheduling algorithm
2. Easy to program
3. First come first served

Disadvantages of FCFS:

1. It is a Non-Preemptive CPU scheduling algorithm, so after the process has been allocated to the CPU, it will never release the CPU until it finishes executing.
2. The Average Waiting Time is high.
3. Short processes that are at the back of the queue have to wait for the long process at the front to finish.
4. Not an ideal technique for time-sharing systems.
5. Although it is easy to implement, but it is poor in performance since the average waiting time is higher as compare to other scheduling algorithms.

Here we have simple formulae for calculating various times for given processes:

Turn Around Time: Time taken to complete after arrival. In simple words, it is the difference between the Completion time and the Arrival time.

Waiting Time: Total time the process has to wait before it's execution begins. It is the difference between the Turn Around time and the Burst time of the process.

For the program below, we have considered the arrival time to be 0 for all the processes, try to implement a program with variable arrival times.

Algorithm:

Step1: Create the number of processes.

Step2: Get the Process ID and Burst time for each process.

Step3: Initially, waiting time of first process is zero and Total time for the first process is the starting time of that process.

Step4: Calculate the Completion Time for the remaining processes.

Step5: Waiting time of one process is the difference between the Turn Around time and the Burst time of the process.

Step6: Turn Around Time of process is calculated by adding Waiting time and Burst time.

Step7: Calculate Average waiting time by dividing the total waiting time by total number of processes.

Step8: Calculate Average turn around time by dividing the total time by the number of processes.

Step9: Display the result.

Program:

```
#include <stdio.h>

#include <conio.h>

void main()
{
    int n,i,bt[100],wt=0,tat=0;
    float avg_wt,avg_tat;
    clrscr();
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter the burst time of process[%d]",i+1);
        scanf("%d",&b[i]);
    }
    printf("\nPid\t\tBT\t\t WT\t\t TAT");
    for(i=0;i<n;i++)
    {
        tat=bt[i]+wt;
        printf("\n%d\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt,tat);
        avg_wt=avg_wt+wt;
        avg_tat=avg_tat+tat;
        wt=wt+bt[i];
    }
    avg_wt=avg_wt/n;
    avg_tat=avg_tat/n;
```

```
printf("\nAverage waiting time is : %f",avg_wt);  
printf("\nAverage turn around time is : %f",avg_tat);  
getch();  
}
```

Output:

Enter the number of processes: 3
Enter the burst time of process[1] = 4
Enter the burst time of process[2] = 2
Enter the burst time of process[3] = 3

Pid	BT	WT	TAT
1	4	0	4
2	2	4	6
3	3	6	9

Average waiting time is = 3.333333

Average Turn around time is = 6.333333

Conclusion :

Thus the C program to implement CPU scheduling algorithm for first come first serve was executed successfully.

Expt No: 5 (b)

Title : Program based on Non-Preemptive CPU Scheduling Algorithms..

Aim : Write a C program to implement Shortest Job First (SJF) scheduling algorithm.

Description :

Shortest Job First (SJF) scheduling algorithm, is a scheduling policy that selects the waiting process with the smallest execution time to execute next. It is practically infeasible as Operating System may not know burst times and therefore may not sort them. While it is not possible to predict execution time, several methods can be used to estimate the execution time for a job, such as a weighted average of previous execution times. SJF can be used in specialized environments where accurate estimates of running time are available.

Advantages of SJF:

1. Maximum throughput.
2. Shortest jobs are favoured.
3. Minimum waiting and turn around time as compared with other scheduling algorithms.

Disadvantages of SJF:

1. It may cause starvation if shorter process keeps coming.
2. Very hard to implement as Operating System **cannot calculate the burst time** of every process and will not be able to sort the processes.
3. It cannot be implemented at the level of short-term CPU scheduling.

Algorithm:

Step1: Get the number of process.

Step2: Get the Process ID and Burst time for each process.

Step3: Initially the waiting time of first short process as 0 and total time of first short process is the service time of that process.

Step4: Calculate the total time and waiting time of remaining process.

Step5: Waiting time of one process is the difference between the Turn Around time and the Burst time of the process.

Step6: Turn Around Time of process is calculated by adding Waiting time and Burst time.

Step7: Calculate Average waiting time by dividing the total waiting time by total number of process.

Step8: Calculate Average turn around time by dividing the total time by the number of process.

Step9: Display the result.

Program:

```
#include <stdio.h>
#include <conio.h>
void main()
{
    int n,bt[100],wt=0,tat=0;
    int i,j,temp;
    float avg_wt,avg_tat,awt=0,atat=0;
    clrscr();
    printf("Enter number of processes: ");
    scanf("%d",&n);
    for(i=0;i<n;i++)
    {
        printf("Enter the burst time of process[%d]",i+1);
        scanf("%d",&b[i]);
    }
    for(i=0;i<n;i++)
    {
        for(j=0;j<n;j++)
        {
            if(bt[i]<bt[j])
            {
```

```
temp=bt[i];
bt[i]=bt[j];
bt[j]=temp;
}
}
}
printf("\nPid\t\tBT\t\tWT\t\tTAT");
for(i=0;i<n;i++)
{
tat=bt[i]+wt;
printf("\n%d\t\t%d\t\t%d\t\t%d",i+1,bt[i],wt,tat);
awt=awt+wt;
atat=atat+tat;
wt=wt+bt[i];
}
avg_wt=awt/n;
avg_tat=atat/n;
printf("\nAverage waiting time is : %f",avg_wt);
printf("\nAverage turn around time is : %f",avg_tat);
getch();
}
```

Output

Enter the number of process : 3
Enter the burst time of process[1]= 11
Enter the burst time of process[1]= 5
Enter the burst time of process[1]= 20

Pid	BT	WT	TAT
1	5	0	5
2	11	5	16
3	20	16	36

Average waiting time is =7.000000

Average Turnaround time is = 19.000000

Conclusion :

Thus the C program to implement CPU scheduling algorithm for Shortest Job First was executed successfully.

Procedure:

1. Install Microsoft Windows and the Turbo C compiler on your computer.
2. Open Turbo C and create a new file for your program.
3. Ask the user to enter the number of processes they want to schedule.
4. For each process, ask for the, burst time, and priority.
5. Choose which scheduling algorithm to use.
6. Execute the chosen algorithm and display the results.
7. Compare these results to see how each algorithm performs.
8. Discuss the pros and cons of each algorithm based on the results you observed.

Practice Problems on Non-Preemptive CPU Scheduling Algorithms

Task 1: FCFS Scheduling Algorithm

Consider the following set of processes. Execute the FCFS scheduling program and find the Average Waiting Time and Average Turnaround Time.

Process ID	Arrival Time	Burst Time
P1	0	5
P2	1	3
P3	2	8
P4	4	6
P5	6	2

Task 2: FCFS Scheduling Algorithm

Consider the following set of processes. Implement FCFS scheduling and display the output.

Process ID	Arrival Time	Burst Time
P1	2	4
P2	0	7
P3	5	3
P4	6	5
P5	8	2

Find:

1. Average Waiting Time
2. Average Turnaround Time

Task 3: SJF Scheduling Algorithm

Consider the following set of processes. Execute the SJF scheduling program and calculate the average waiting time and average turnaround time.

Process ID	Arrival Time	Burst Time
P1	0	6
P2	2	3
P3	4	8
P4	5	2

Task 4: SJF Scheduling Algorithm

Consider the following set of processes. Execute the SJF scheduling program and find the Average Waiting Time and Average Turnaround Time.

Process ID	Arrival Time	Burst Time
P1	0	9
P2	1	4
P3	3	2

References –

a. Textbook –

- i. "Operating System Concepts", Abraham Silberschatz, Henry Korth, S. Sudarshan- 10th Edition (2018), Publisher: Wiley
- ii. "Operating Systems: Principles and Practice"- Thomas Anderson and Michael Dahlin, 2nd Edition (2018), Publisher: Recursive Books

b. Online references –

- i. <https://www.geeksforgeeks.org/cpu-scheduling-set-1-introduction/>
- ii. https://www.tutorialspoint.com/operating_system/os_scheduling_algorithms.htm
- iii. <https://github.com/search?q=cpu+scheduling+notebook>

Expected Oral Questions –

1. What is the main objective of CPU scheduling in an operating system?
2. Explain the First-Come, First-Served (FCFS) scheduling algorithm and its key characteristics?
3. How does the Shortest Job First (SJF) scheduling algorithm work, and what are its advantages and disadvantages?
4. Explain how the waiting time and turnaround time are calculated for processes in a scheduling algorithm.
5. Discuss the concept of starvation in CPU scheduling and which algorithms are prone to causing it?
6. How does a preemptive scheduling algorithm differ from a non-preemptive scheduling algorithm?
7. How can the implementation of a CPU scheduling algorithm affect overall system performance and responsiveness?
8. What is the role of context switching in CPU scheduling, and how does it impact system efficiency?

